Project: Vibe Navigator

Version: 4.1.0 **Last Updated:** July 29, 2025

1. Project Overview & Vision

1.1 The Problem

In a world saturated with streaming services and content platforms like Netflix, Spotify, and YouTube, users often face "decision fatigue". The core issue is not a lack of choice, but an overwhelming abundance of it , leading users to spend more time searching for content than enjoying it.

1.2 The Solution

Vibe Navigator is a "zero-click" mobile application that serves as a decisive entertainment advisor. Its primary function is to eliminate decision fatigue by providing a single, highly curated media recommendation the moment the app is opened. It achieves this by deeply understanding a user's unique taste through

Content-Based Filtering and then applying real-world context (weather, time, location) to select the perfect suggestion.

1.3 Target Audience

The application is designed for individuals who subscribe to multiple streaming services, value their time, and are often overwhelmed by choice. This includes busy professionals, students, and anyone who prefers immediate, mood-appropriate entertainment without the effort of Browse.

---

2. Core Features

- **Content-Driven Personalization**
    - **Description:** The app builds a detailed "Taste Profile" for each user based on the content they like. It recommends new items that share similar attributes (genres, themes, keywords) to items the user has previously enjoyed.
    - **User Story:** "As a user who loves sci-fi movies with complex plots, I want the app to learn this and recommend other movies with similar themes, not just generic blockbusters."
- **Automatic Context Analysis**

    - **Description:** The application automatically uses the phone's time, weather, and location data to understand the user's current environment and apply a final layer of context to recommendations.
    - **User Story:** "As a user, even though I like action movies, I want the app to know not to recommend a loud blockbuster on a quiet Sunday morning."
- **User Interaction & Feedback Loop**
    - **Description:** Users can actively train their profile by rating recommendations. This feedback continuously refines their Taste Profile, making future suggestions even more accurate.
    - **User Story:** "As a user, if I get a recommendation I love, I want to be able to 'like' it so the app knows to give me more suggestions like that one."
- **Community & Social Groups**
    - **Description:** Users can create and join public or private groups centered around specific movie genres. Within these groups, members can share movie suggestions and discuss them.
    - **User Story:** "As a user who is a huge fan of horror movies, I want to join a 'Horror Fans' group to discover hidden gems and share my own favorite scary movies with like-minded people."
- **Unified Media Library & Deep Linking**
    - **Description:** The system recommends content across various media types, including movies, TV series, music, podcasts, and news, from different platforms. Every recommendation includes a direct link that opens the content in the native streaming app.

---

3. Technology Stack

The project uses a modern, full-stack architecture focusing on robust and scalable technologies, all of which can operate within a free tier.

Component	Technology	Detailed Reason for Choice
Mobile App	React Native	Allows for a single codebase to build and deploy native applications on both iOS and Android, maximizing development efficiency.

Component	Technology	Detailed Reason for Choice	
Backend Server	**Java (Spring Boot)**		**Primary Choice:** A professional-grade, high-performance, and robust framework. Its strong typing and security features are ideal for the core business logic.
Database	PostgreSQL	A powerful, open-source object-relational database renowned for its reliability and strong support for structured user data.	
Caching	Redis	A high-performance, in-memory data store perfect for caching computed Taste Profiles and external API responses, ensuring low-latency recommendations.	
Search/Vector DB	Elasticsearch	Its powerful text analysis and "more like this" query capabilities are excellent for implementing content-based similarity search.	
Movie/TV API	TMDb	A free, comprehensive API for movie and TV series metadata, including posters and ratings, which is essential for content analysis.	
Music/Podcast API	Spotify API	Provides free developer access to its vast music and podcast catalog, enabling recommendations and discovery.	

Component	Technology	Detailed Reason for Choice
News API	YouTube Data API v3	Offers a free, robust way to search for news videos from reputable channels, filter by category, and get necessary metadata.

Export to Sheets

Polyglot Microservices: Choosing the Right Tool for the Job

While Java is the backbone of this project, a key advantage of a microservices architecture is the ability to use the best technology for each specific job.

- **For the Taste Profile Builder: Python**
    - **Reasoning:** The Taste Profile Builder performs complex data processing and mathematical calculations (vectorization). **Python** is the industry leader in data science, and its libraries (Pandas, scikit-learn) make these operations far simpler and more efficient to write than in Java.
- **For Vibe Sync (Real-time Service): Node.js**
    - **Reasoning:** The Vibe Sync feature requires managing thousands of persistent WebSocket connections. **Node.js's** non-blocking, event-driven architecture was designed for this exact use case, making it more resource-efficient for this specific task.

---

4. System Architecture Explained

The architecture is designed as a distributed system to handle data processing and recommendation serving separately, ensuring scalability.

- **Client (React Native):** The user interface. It sends user interaction data and context (GPS) to the backend and displays the final recommendation.
- **API Gateway:** The single, secure entry point for all client requests. It handles authentication (validating JWTs) and routes traffic to the appropriate internal microservice.
- **Backend Services (Java Spring Boot):**
    - **User & Profile Service:** Manages user accounts, authentication, and explicit preferences.

    - **Interaction Service:** Receives all user interactions (e.g., like, dislike), stores them, and publishes an event to Kafka.
    - **Taste Profile Builder (Kafka Consumer):** A background service that listens for new user interactions. It re-calculates the user's "Taste Profile" vector and saves it to Redis.
    - **Recommendation Service:** Generates recommendations by fetching the Taste Profile from Redis and using Elasticsearch to find similar items.
- **Data Stores & Message Queue:**
    - **PostgreSQL:** The primary database for user accounts and the raw log of all user interactions.
    - **Kafka:** Acts as a durable, asynchronous message queue between the Interaction Service and the Taste Profile Builder.
    - **Redis:** A high-speed cache for storing the computed Taste Profile vectors.
    - **Elasticsearch:** Stores "Item Profiles" for all media to enable fast similarity searches.

---

5. The Content-Based Recommendation Engine Explained

This engine works by matching the "content" of items to a user's unique taste profile.

- **Item & User Profiles (Vectorization):** Every piece of media is transformed into a numerical vector based on its attributes.
    - **Movie/Series:** Attributes include genres, keywords, director, actors.
    - **Music:** Attributes include genre, artist_style, tempo, energy, danceability.
    - **Podcast/News:** Attributes include category, host/channel, and keywords from the description.
    - A user's "Taste Profile" is a dynamically computed vector representing the weighted average of all items they have liked.
- **Recommendation Flow:**
    1. **Trigger:** The user opens the app, sending their context (GPS, time) to the Recommendation Service.
    2. **Fetch Profile:** The service instantly retrieves the user's pre-computed Taste Profile vector from **Redis**.

3. **Media Type Decision:** The engine uses a ruleset to decide which media type to prioritize. For example, morning commute times might boost the score for "podcast" and "news," while late evenings boost "movie" or "series".
4. **Contextual Pre-Filtering:** The service uses context (e.g., "rainy night") to apply a broad filter to the chosen media catalog in **Elasticsearch**.
5. **Similarity Scoring (Cosine Similarity):** For each item in the filtered pool, the engine calculates the **Cosine Similarity** between the item's vector and the user's taste vector.
6. **Selection:** The item with the highest similarity score is chosen as the final recommendation.

---

6. Modules & API Design

The backend is organized into the following modules.

Module 1: Authentication (High Priority)

- **Description:** This module handles user identity, signup, login, and secure access to the application. It is the foundation of all personalized features.
- **API Endpoints:** POST /auth/register ,

POST /auth/login ,

GET /auth/verify-email ,

POST /auth/request-password-reset.

Module 2: Profile Management (High Priority)

- **Description:** Manages all user-specific preferences that power the recommendation engine's personalization.
- **API Endpoints:** GET /profile ,

PUT /profile.

Module 3: Interaction & Taste Profile (High Priority)

- **Description:** The core of the learning system. It captures user feedback and triggers the background process to update their taste profile.
- **API Endpoints:**

    - **POST /interactions**: The client sends user feedback here (e.g., like, dislike). The service saves the interaction and publishes an event to Kafka.

Module 4: Recommendation Engine (High Priority)

- **Description:** The read-only service responsible for generating the final recommendation using the pre-computed taste profiles.
- **API Endpoints:**
    - **POST /recommendations/now**: The primary endpoint. It accepts the user's context and returns a single structured recommendation object.

Module 5: Vibe Sync (Group Recommendation) (Advanced)

- **Description:** A real-time social feature that allows a group of friends to get a single, shared recommendation based on their combined tastes.
- **Advanced Tech Showcase: WebSockets** for the real-time lobby, **gRPC** for high-performance internal communication, and **Redis** to store the state of active lobbies.
- **API Endpoints:** POST /sync/create, POST /sync/join.
- **WebSocket Endpoint:** /ws/sync/{lobby_code} for managing the live session.

Module 6: Community Groups (Clubs) (Advanced)

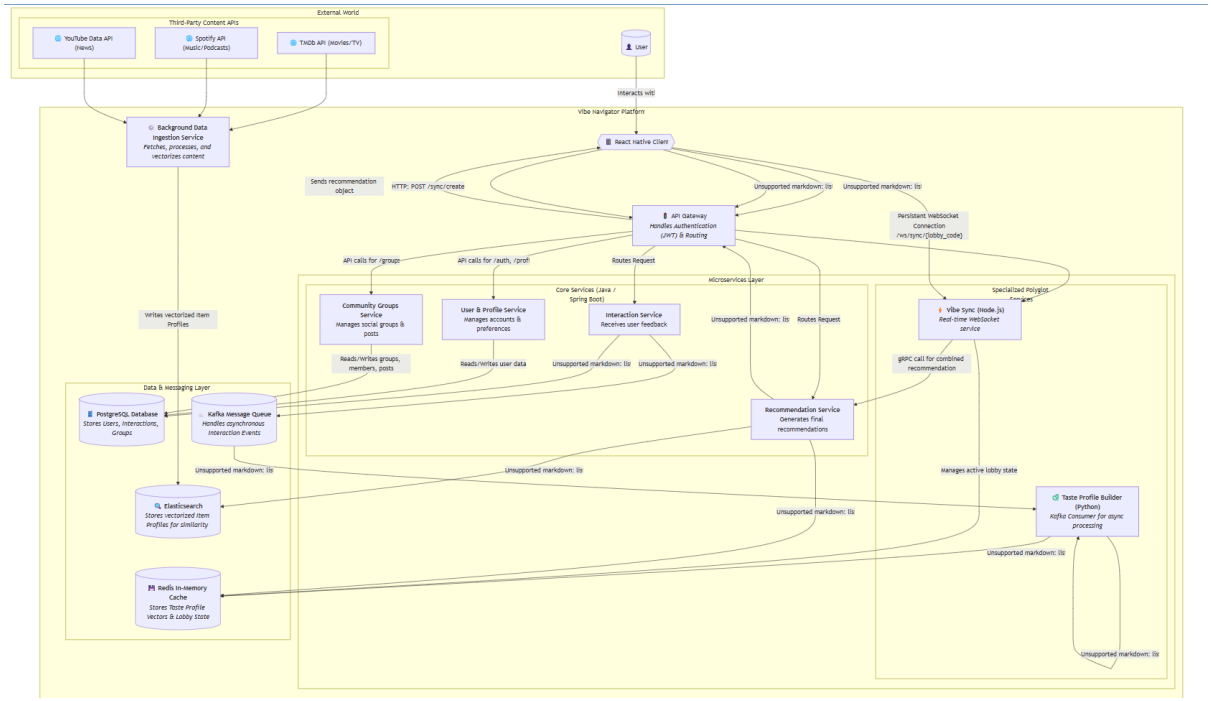
- **Description:** A social feature that allows users to form communities around specific movie genres, share recommendations, and discuss them.
- **Features:** Create public/private groups, group discovery based on user taste, join/request to join groups, and post movie feedback within a group.
- **API Endpoints:**
    - POST /groups: Create a new group (public or private).
    - GET /groups/recommendations: Recommends relevant groups to the user based on their Taste Profile.
    - POST /groups/{groupId}/join: Allows a user to join a public group or request to join a private group.
    - POST /groups/{groupId}/requests/{requestId}/approve: (Admin only) Approve a pending join request.
    - GET /groups/{groupId}/posts: View all movie feedback posts within a group.

- POST /groups/{groupId}/posts: Create a new movie feedback post within a group.

---

7. Development Sprints

- **Sprint 1: The Core MVP & Foundation:** Set up projects, get API keys, and build the non-personalized movie recommendation flow.
- **Sprint 2: User Personalization & Onboarding:** Add user accounts, preferences, and JWT authentication.
- **Sprint 3: Building the Content-Based Engine:** Implement the core logic for vectorization and similarity search in Elasticsearch.
- **Sprint 4: Interaction & Profile Adaptation:** Build the Interaction Service and Taste Profile Builder with Kafka to enable the feedback loop.
- **Sprint 5: Expanding Media Types:** Integrate the Spotify API for music/podcasts and the YouTube API for news. Enhance the engine to handle these new content types.
- **Sprint 6 (Advanced): Social Vibe Sync:** Build the real-time group recommendation feature with WebSockets.
- **Sprint 7 (Advanced): Community Groups:** Build the new group/club feature for social interaction.
- **Sprint 8 (Polish): Refine UI/UX:** Add settings pages, recommendation history, and fluid animations to polish the app.



Spring Boot Dependencies for Vibe Navigator

When you're on the Spring Initializr website, add the following dependencies:

- **Spring Web:** The foundation for creating your REST APIs (like POST /auth/login and GET /profile).
- **Spring Security:** Essential for handling user authentication, registration, and securing your API endpoints with JWTs.
- **Spring Data JPA:** The standard way to manage and interact with your user data in the **PostgreSQL** database.
- **PostgreSQL Driver:** The specific driver needed to connect your application to the PostgreSQL database.

- **Spring for Apache Kafka:** Needed to publish and listen for events, which is the core of the asynchronous Interaction Service.
- **Spring Data Redis:** Allows your application to connect to **Redis** for high-speed caching of the computed "Taste Profile" vectors.
- **Spring Data Elasticsearch:** Required to connect to and perform similarity searches in **Elasticsearch**, which powers the recommendation engine.
- **Lombok** (optional but recommended): A very common tool to reduce boilerplate code (like getters, setters, and constructors) in your Java classes.
- **Spring Boot DevTools** (optional but recommended): Useful for development, enabling features like automatic server restarts when you change code.

For taste-profile-builder-python/README.md

Create a file named README.md inside the taste-profile-builder-python directory and paste the following content into it:

Taste Profile Builder (Python Service)

1. Overview

[cite_start]This service is a background Kafka consumer responsible for processing user interactions (like 'like' or 'dislike') and updating their "Taste Profile" vector[cite: 78, 79]. [cite_start]It performs the complex data processing and mathematical calculations required for the content-based recommendation engine[cite: 62].

[cite_start]Python was chosen because it is the industry leader in data science, and its libraries make vectorization and other data operations highly efficient[cite: 63].

2. Setup Instructions

These steps will set up the project environment.

1. **Navigate to this directory:**

   ```bash
   cd taste-profile-builder-python
   ```

2. **Create a Python virtual environment:**

   ```bash
   python -m venv venv
   ```

3. **Activate the virtual environment:**

   * **Windows:** `.\venv\Scripts\activate`

   * **macOS/Linux:** `source venv/bin/activate`

4. [cite_start]**Create a `requirements.txt` file** with the following dependencies mentioned in the project plan[cite: 63]:

   ```txt
   pandas
   scikit-learn
   kafka-python
   ```

5. **Install the dependencies:**

   ```bash
   pip install -r requirements.txt
   ```

3. Running the Service

The service will be started by running the main Python script, which will contain the Kafka consumer logic.

```bash
Python main.py
```

Vibe Sync (Node.js Service) README

- **Overview**
    - This service provides the real-time "Vibe Sync" feature, which allows a group of users to get a shared recommendation. It is specifically designed to manage thousands of persistent WebSocket connections for the real-time lobbies.
    - Node.js was chosen for its non-blocking, event-driven architecture, which is highly resource-efficient for this exact use case.
- **Setup Instructions**
    - **Navigate to directory:**

Bash

```
cd vibe-sync-service-node
```

    - **Initialize project:**

Bash

```
npm init -y
```

    - **Install dependencies:**

Bash

```
npm install express ws
```

        - express: A minimal and flexible web application framework.
        - ws: A popular, high-performance WebSocket library.
- **Running the Service**

- The service can be started by running the main entry file (e.g., index.js).
- It's recommended to add a start script to package.json:

JSON

```
"scripts": {
  "start": "node index.js"
}
```

- Run the service with the following command:

Bash

```
npm start
```